Interval Taylor Evaluation of $yH[y_u]$

Mathematical estimates, Julia implementation, and Lean statements

Repository snapshot: September 9, 2026

Abstract

This independent note explains how the term $P = yH[y_u]$ is evaluated in the two-million-point Crunchy2 residual calculation. It develops the local derivative enclosures, the seventh-order remainder bound, the role of bound tightening, and the signed assembly of the residual. The relevant Lean statements are presented as ordinary mathematical theorems with proofs. The distinction between analytic theorems, hypotheses about numerical enclosures, and the numerical assumptions of the dedicated certificate is kept explicit. No part of the note requires another manuscript or external LaTeX input file.

1 The function and the certified inputs

Let $a_0, \dots, a_d \in \mathbb{R}$ be the fixed coefficients of the candidate and set

$$y(u) = \frac{a_0}{2} + \sum_{k=1}^{d} a_k \cos(ku), \qquad Q(u) = H[y_u](u), \qquad P(u) = y(u)Q(u). \tag{1.1}$$

The periodic Hilbert transform has the convention

$$H[1] = 0, \qquad H[\cos(ku)] = \sin(ku), \qquad H[\sin(ku)] = -\cos(ku) \quad (k \geq 1). \tag{1.2}$$

In particular $P = yH[y_u]$ is a pointwise product. It is different from the other residual term $H[yy_u]$.

The run documented in `wave-cap/BabenkoRigorous/results/phase4_scalar_n150_20260827_2m_20260829/README.md` uses the following parameters:

Quantity	Value
Fourier degree d	150
Number of coefficients	151
Number of grid points n	2,000,000
Taylor order M	6
Arb working precision	128 bits
Maximum rows per shard	1024

Throughout, put

$$h = \frac{2\pi}{n}, \qquad u_j = (j-1)h \quad (1 \leq j \leq n). \tag{1.3}$$

The cells $[u_j, u_j + h]$ cover $[0, 2\pi]$, including its right endpoint. The estimates use the full spacing h, not $h/2$.

The exact candidate is the one specified by the finite stored numerical inputs, with their exact dyadic interpretation. The certificate is not a claim about an unknown exact solution whose coefficients merely round to these inputs. Arb supplies outward enclosures of all subsequent arithmetic, including π, the grid locations, and trigonometric evaluations.

Theorem 1.1 (Fourier identification and constant-mode convention)**.** *For the function in* (1.1)*,*

$$y_u(u) = -\sum_{k=1}^{d} k a_k \sin(ku), \qquad Q(u) = \sum_{k=1}^{d} k a_k \cos(ku). \tag{1.4}$$

If $\alpha_0 = a_0/2$ and $\alpha_k = a_k$ for $k \geq 1$, the full-constant-mode sum $\sum_{k=0}^{d} \alpha_k \cos(ku)$ equals y. Halving only the zeroth coefficient changes neither the derivative sum nor its Hilbert transform.

Proof. Differentiate the finite sum term by term. Apply linearity of H and (1.2) to each sine term to obtain $H[-ka_k \sin(ku)] = ka_k \cos(ku)$. The coefficient conversion follows from $\cos(0u) = 1$. Both the derivative and the transformed derivative multiply the zeroth coefficient by 0, so neither depends on its value. □

Lean correspondence. `periodicHilbert_yu_eq_Hyu` in `HilbertOnTrigPoly.lean`.

The constant-mode statements correspond to `yHalfDC_apply`, `yu_halfDCCoeffs`, and `Hyu_halfDCCoeffs` in `HalfDCConvention.lean`. Lean's `y`, `yu`, and `Hyu` are defined in `Definitions.lean`; its coefficient vector has length $d + 1 = 151$. The generated half-DC certificate already makes the required zeroth-coefficient conversion.

The product P is a finite trigonometric polynomial of degree at most $2d = 300$, by the cosine product identity. The Julia path below forms its derivative data without projecting it back to degree 150. In particular, this term requires neither an FFT nor an unresolved-mode estimate.

2 Interval Taylor records and the input bounds

Definition 2.1 (Valid Taylor record)**.** For $f \in C^{M+1}(\mathbb{R})$, a valid Taylor record on a grid consists of a positive spacing h, centers $c_{j,m}$, nonnegative errors $e_{j,m}$ for $0 \leq m \leq M$, and global bounds $B_m(f)$ for $0 \leq m \leq M + 1$, such that

$$|f^{(m)}(u_j) - c_{j,m}| \leq e_{j,m}, \qquad 0 \leq m \leq M, \tag{2.1}$$

$$|f^{(m)}(u)| \leq B_m(f), \qquad u \in \mathbb{R}, \quad 0 \leq m \leq M + 1. \tag{2.2}$$

Write $I_{j,m}(f) = [c_{j,m} - e_{j,m}, c_{j,m} + e_{j,m}]$ and

$$U_{j,m}(f) = \sup\{|x| : x \in I_{j,m}(f)\} = |c_{j,m}| + e_{j,m}. \tag{2.3}$$

This is the mathematical content of `TaylorSeriesData` and `IsTaylorSeriesOf` in `TaylorSeriesCorrectness.lean`. Lean uses grid indices $0, \ldots, n-1$; Julia uses $1, \ldots, n$. Julia stores the whole Arb ball in `derivs[j,m+1]`, while Lean's abstract record separates its center and error. These are raw derivatives, not coefficients divided by $m!$. There are seven local derivative columns for $M = 6$, but eight global bound entries, covering derivative orders $0, \ldots, 7$.

An entry of Julia's `sup_bounds` is also an Arb value. It represents a rigorously evaluated bound expression. Its upper endpoint is a usable scalar upper bound; the ball is not asserted to enclose the exact supremum norm. In the formulas below, scalar bounds describe the mathematical inequalities. Julia implements the same bound expressions with outward rounding; no claim of exact floating-point equality with an idealized real-arithmetic formula is needed.

Theorem 2.2 (Coefficient bounds and exact Taylor data)**.** *For* $f(u) = \sum_{k=0}^{d}(\alpha_k \cos(ku)+b_k \sin(ku))$, *define*

$$C_m(f) = \sum_{k=0}^{d} k^m(|\alpha_k| + |b_k|), \qquad 0^0 := 1. \tag{2.4}$$

Then $|f^{(m)}(u)| \le C_m(f)$ *for all* $u \in \mathbb{R}$ *and all* $m \ge 0$*. Consequently, a record with exact derivative samples, zero errors, these global bounds, and positive spacing is a valid Taylor record. For the factors* y, Q *in* (1.1)*, valid bounds are*

$$B_0(y) = \frac{|a_0|}{2} + \sum_{k=1}^{d} |a_k|, \qquad B_m(y) = \sum_{k=1}^{d} k^m |a_k| \quad (m \ge 1), \tag{2.5}$$

$$B_m(Q) = \sum_{k=1}^{d} k^{m+1}|a_k| \qquad (m \ge 0). \tag{2.6}$$

Proof. Each differentiation of a nonconstant sine or cosine multiplies its coefficient by k and changes only its sign and trigonometric type. Since sine and cosine have absolute value at most one, the triangle inequality gives the asserted sum. Constant terms vanish after one differentiation; at order zero the convention $0^0 = 1$ supplies a valid bound, even for the identically zero sine term. Finite trigonometric sums are smooth, so exact samples with zero errors satisfy (2.1), and the coefficient estimate supplies (2.2). Substitute $\alpha_0 = a_0/2$, $\alpha_k = a_k$, and the coefficients of Q to obtain (2.5)–(2.6). □

Lean correspondence. `trigpoly_to_taylor_correct` in `TaylorSeriesCorrectness.lean`.

This Lean theorem assumes the identities giving the exact derivative samples, zero sample errors, and the displayed coefficient-bound formula. It is not a proof that the finite-precision Arb output has zero error. In the numerical pipeline, inclusion of the exact derivative in the computed ball is supplied by the interval evaluation instead.

Julia's `trigpoly_to_taylor` forms coefficient derivatives through order $M + 1$, evaluates orders $0, \dots, M$ at the grid using `tp_eval`, and applies `supnorm_bound` to all orders $0, \dots, M + 1$. Thus a seventh-order bound is available without sampling seventh derivatives on the grid. The factors y and Q are enclosed separately at each grid point, but no separate whole-cell supremum bound is taken before their local derivative records are multiplied.

3 Leibniz multiplication and its formal statements

Theorem 3.1 (Leibniz derivative bound)**.** *Suppose* f, g *are* C^m *near* u*, and* $|f^{(k)}(u)| \le F_k$, $|g^{(k)}(u)| \le G_k$ *for* $0 \le k \le m$*. Then*

$$|(fg)^{(m)}(u)| \le \sum_{k=0}^{m} \binom{m}{k} F_k G_{m-k}. \tag{3.1}$$

If the factor bounds hold globally, the same expression is a global bound.

Proof. Repeated differentiation gives $(fg)^{(m)} = \sum_{k=0}^{m} \binom{m}{k} f^{(k)} g^{(m-k)}$. Take absolute values, apply the triangle inequality, and bound each factor. The binomial coefficients and the factor bounds are nonnegative. The argument holds at every point when the hypotheses hold globally. □

Lean correspondence. `abs_iteratedDeriv_mul_le` in `LeibnizBound.lean`.

The named Lean theorem is the pointwise statement; the global version is its immediate application at every u.

For $P = yQ$, Julia constructs

$$I_{j,m}(P) = \sum_{k=0}^{m} \binom{m}{k} I_{j,k}(y) I_{j,m-k}(Q), \qquad 0 \le m \le 6, \tag{3.2}$$

$$L_m(P) = \sum_{k=0}^{m} \binom{m}{k} B_k(y) B_{m-k}(Q), \qquad 0 \le m \le 7. \tag{3.3}$$

The interval operations in (3.2) round outwards. For example, the order-two row uses the three terms $I_{j,2}(y)I_{j,0}(Q) + 2I_{j,1}(y)I_{j,1}(Q) + I_{j,0}(y)I_{j,2}(Q)$. By interval inclusion and the Leibniz identity, this encloses $P''(u_j)$. Equation (3.3), evaluated rigorously, supplies the initial global bound vector. In particular its top entry includes all eight terms of the seventh-order Leibniz sum.

Theorem 3.2 (Correctness of a product Taylor record)**.** *Let valid records for $f, g \in C^{M+1}(\mathbb{R})$ have the same grid, with centers $c^f_{j,k}, c^g_{j,k}$ and bounds $B_k(f), B_k(g)$. Define*

$$p_{j,m} = \sum_{k=0}^{m} \binom{m}{k} c^f_{j,k} c^g_{j,m-k}. \tag{3.4}$$

Suppose numbers $\eta_{j,m}$ satisfy $|(fg)^{(m)}(u_j) - p_{j,m}| \le \eta_{j,m}$ for all $1 \le j \le n$ and $0 \le m \le M$. Then the centers $p_{j,m}$, errors $\eta_{j,m}$, and global bounds $\sum_{k=0}^{m} \binom{m}{k} B_k(f) B_{m-k}(g)$ define a valid record for fg.

Proof. Products preserve C^{M+1} regularity. The assumed error inequality is exactly the sample-validity requirement. Theorem 3.1 supplies the global bounds at each order. The common spacing remains positive. □

Lean correspondence. `mul_taylorseries_correct` in `TaylorSeriesCorrectness.lean`.

The inequality for η is the explicit Lean hypothesis `h_err_valid`; Lean does not infer the production Arb errors here.

To see how a sufficient error can be obtained, write exact input samples as $c^f + \delta_f$ and $c^g + \delta_g$, with $|\delta_f| \le e^f$, $|\delta_g| \le e^g$. Expanding one product gives

$$|(c^f + \delta_f)(c^g + \delta_g) - c^f c^g| \le |c^f| e^g + |c^g| e^f + e^f e^g. \tag{3.5}$$

Summing these expressions with binomial weights gives a sufficient $\eta_{j,m}$ for the exact center $p_{j,m}$. If the computed center is $\widehat{p}_{j,m}$ and $|\widehat{p}_{j,m} - p_{j,m}| \le \rho_{j,m}$, add $\rho_{j,m}$ to the error. Julia's interval operations enclose both input uncertainty and arithmetic rounding. This explanatory error formula is not an assertion that Julia stores precisely that radius.

Theorem 3.3 (Existence of exact records)**.** *Every 2π-periodic C^{M+1} function has a valid exact-sample record on any prescribed finite grid of positive spacing. Given valid records for f and g on a common grid, there also exists a valid record for fg with exact samples, zero sample errors, and the Leibniz global bounds.*

Proof. Each derivative through order $M+1$ of a periodic C^{M+1} function is continuous and periodic. Its absolute value attains a maximum on $[0, 2\pi]$, and periodicity extends this bound to $\mathbb{R}$. Take the true derivatives as the centers and zero errors. For the second assertion, take the true derivatives of fg as centers and use Theorem 3.1 for the supplied global bounds. □

Lean correspondence. `exists_isTaylorSeriesOf_of_contDiff_periodic` in `CertificateAxioms.lean`.

The specialized grid-spacing theorem is `cert_arb_trigpoly_to_taylor`; the product existence theorem is `cert_arb_ts_mul`. Despite their names and their containing file, these declarations are theorems, not numerical axioms. Their existential conclusions do not identify the witness with the particular arrays produced by Crunchy2.

4 From a Taylor record to a continuum estimate

Theorem 4.1 (Taylor remainder on a cell)**.** *Let $f \in C^{M+1}$ on a neighborhood of $[v, v+h]$, $h > 0$, and suppose $|f^{(M+1)}(u)| \le B$ there. For $0 \le t \le h$,*

$$\left| f(v+t) - \sum_{m=0}^{M} f^{(m)}(v) \frac{t^m}{m!} \right| \le B \frac{t^{M+1}}{(M+1)!}. \tag{4.1}$$

If $f^{(m)}(v) \in I_m$, then

$$f(v+t) \in \sum_{m=0}^{M} I_m \frac{t^m}{m!} + \left[-B \frac{t^{M+1}}{(M+1)!}, B \frac{t^{M+1}}{(M+1)!} \right]. \tag{4.2}$$

Proof. At $t = 0$ the remainder is zero. For $t > 0$, Taylor's theorem in Lagrange form gives a point $\xi \in (v, v+t)$ for which the remainder is $f^{(M+1)}(\xi) t^{M+1}/(M+1)!$. Apply the derivative bound. The interval statement follows by enclosing each derivative value in I_m and adding the symmetric remainder interval. □

Lean correspondence. `abs_taylor_remainder_le` in `TaylorRemainder.lean`.

The Lean statement uses derivatives within the closed interval and a Taylor polynomial defined with these derivatives. For the globally smooth functions considered here they agree with the usual derivatives, giving the displayed specialization. This avoids any endpoint regularity issue.

Theorem 4.2 (Grid-to-continuum bound)**.** *Let a valid Taylor record for f be given on the grid* (1.3)*. Then*

$$\sup_{u \in [0, 2\pi]} |f(u)| \le E(f) := \max_{1 \le j \le n} \left\{ \sum_{m=0}^{M} U_{j,m}(f) \frac{h^m}{m!} + B_{M+1}(f) \frac{h^{M+1}}{(M+1)!} \right\}. \tag{4.3}$$

For periodic f, the same bound holds on all of $\mathbb{R}$.

Proof. For any $u \in [0, 2\pi]$, choose a cell with $u = u_j + t$ and $0 \le t \le h$; at $u = 2\pi$ use the last cell. Apply Theorem 4.1, the triangle inequality, and $|f^{(m)}(u_j)| \le U_{j,m}(f)$. Since $0 \le t \le h$, replacing t^m by h^m increases each nonnegative contribution. Taking the maximum over j proves the claim. Periodicity reduces any real argument to a representative in one period. □

Lean correspondence. `grid_to_supNorm_taylor` in `GridToSupNormTaylor.lean`.

The record-level version is `sup_bound_taylorseries_correct` in `TaylorSeriesCorrectness.lean`; it assumes that the displayed cell expression is at most a supplied number for every j.

For P and $M = 6$, the local estimate is therefore

$$|P(u_j + t)| \le \sum_{m=0}^{6} U_{j,m}(P) \frac{h^m}{m!} + B_7(P) \frac{h^7}{7!}, \qquad 0 \le t \le h. \tag{4.4}$$

This explains the continuum meaning of the product record. The production calculation does not scan P separately for the final residual certificate; it uses the record in the signed assembly described in Section 6.

5 What bound tightening does, and where it is used

Lemma 5.1 (One grid-and-Lipschitz step)**.** *Let $f \in C^{m+1}(\mathbb{R})$, suppose $|f^{(m+1)}| \le C$ globally, and let $A_m \ge \max_j |f^{(m)}(u_j)|$. Then*

$$|f^{(m)}(u)| \le A_m + hC \qquad (u \in [0, 2\pi]). \tag{5.1}$$

The same conclusion holds when the derivative bound is known only on the covered interval. If f is periodic, the result extends globally.

Proof. Choose $u = u_j + t$ with $0 \le t \le h$. The mean value inequality for $f^{(m)}$ gives $|f^{(m)}(u) - f^{(m)}(u_j)| \le Ct$. Add the bound at u_j and use $t \le h$. Only values in the cell are used. For periodic f, its derivatives are periodic, so one period suffices. □

Lean correspondence. `grid_lipschitz_sup_bound_step` in `TightenSupBoundsLemma.lean`.

The same file includes the local-interval variant used by the recursion.

Theorem 5.2 (Downward grid-bound recursion)**.** *Suppose $f \in C^{M+1}(\mathbb{R})$, $T_{M+1} \ge \left\|f^{(M+1)}\right\|_\infty$, and $A_m \ge \max_j |f^{(m)}(u_j)|$ for $0 \le m \le M$. Set, downwards,*

$$T_m = A_m + hT_{m+1}, \qquad m = M, M-1, \ldots, 0. \tag{5.2}$$

Then $|f^{(m)}(u)| \le T_m$ on the covered interval for every $m \le M$. If f is periodic, the bounds are global.

Proof. The top-order hypothesis starts the induction. At order m, the bound already proved at order $m+1$, together with Lemma 5.1, gives $A_m + hT_{m+1}$. Induct downwards and then use periodicity. □

Lean correspondence. `tighten_sup_bounds_correct` in `TightenSupBoundsLemma.lean`.

The recursion itself is `tightenSupBoundsRecursion`.

Corollary 5.3 (The production update with a minimum)**.** *Let L_m be valid global bounds and put $A_m = \max_j U_{j,m}(f)$. For periodic f, the downward updates*

$$\widetilde{B}_{M+1} = L_{M+1}, \qquad \widetilde{B}_m = \min\{L_m, A_m + h\widetilde{B}_{m+1}\} \quad (m = M, \ldots, 0) \tag{5.3}$$

are valid global bounds, with the top bound unchanged.

Proof. Induct downwards as in Theorem 5.2. Both entries of the minimum are valid global bounds, so their minimum is also valid. Taking outward upper endpoints preserves validity in interval arithmetic. □

This is a mathematical corollary describing Julia's `_tighten_from_max!`; it is not being identified with a separate Lean declaration of the same algorithm.

Proposition 5.4 (The record transform currently defined in Lean)**.** *Let a valid record for a periodic function have bounds B_m and grid absolute bounds $A_m = \max_j(|c_{j,m}| + e_{j,m})$. Define*

$$\begin{aligned} S_{M+1} &= B_{M+1}, \quad S_m = B_m + hS_{m+1}, \\ \widehat{B}_{M+1} &= B_{M+1}, \quad \widehat{B}_m = \min\{B_m, A_m + hS_{m+1}\}. \end{aligned} \tag{5.4}$$

Keeping the centers, errors, and spacing fixed, and replacing B by $\widehat{B}$, preserves validity of the Taylor record.

Proof. The old global bounds B_m are also bounds at all grid points. Applying Theorem 5.2 with these as its grid bounds proves validity of S_m. Lemma 5.1 with the actual grid bound A_m then proves validity of $A_m + hS_{m+1}$. Take the minimum with the old valid bound. Periodicity gives global validity; all other record conditions are unchanged. □

Lean correspondence. `tighten_sup_bounds_taylorseries_correct` in `TightenSupBoundsLemma.lean`.

Equation (5.4) transcribes the current definition of `TaylorSeriesData.tighten` for a nonempty grid. Its internal recursion uses the old global bounds. It is therefore not identical to Julia's recursion (5.3), which reuses the updated next-order bounds and the per-order grid maxima. Both constructions are valid, but this Lean theorem is not an equality or execution-equivalence proof for the Julia routine.

For $P = yQ$, the product routine calls tightening after constructing (3.2) and (3.3). Its dependency is

$$I_{j,m}(P) \longrightarrow U_{j,m}(P) \longrightarrow A_m(P) \longrightarrow \widetilde{B}_m(P). \tag{5.5}$$

Tightening does not change $I_{j,m}$ or $U_{j,m}$. At $M = 6$ it can improve orders $0, \ldots, 6$, but leaves $B_7(P)$ unchanged.

Proposition 5.5 (No effect of this pass on the final residual scan)**.** *Suppose P is used only additively in the residual, and the final scan is performed at order six. Replacing only $B_0(P), \ldots, B_6(P)$ by tighter values, while preserving its local derivative balls and $B_7(P)$, does not change the data consumed by that scan.*

Proof. Signed addition and subtraction assemble local derivative balls from local derivative balls, without using the global bound vector. The seventh-order residual bound uses the seventh-order bounds of the summands. Both of these inputs are preserved. Formula (4.3) uses no lower-order global bounds. □

This proposition is a dependency observation about the current Julia pipeline, not a separately named Lean theorem. It explains why the generic tightening call is redundant for the final use of this particular product.

It does not imply that grid tightening is unused in the full residual. For example, with $D = (y_u^2 + (1+Q)^2)^{1/2}$ and $C_1 = \partial_u(y_u D^{-1})$, the identity $C_1^{(7)} = (y_u D^{-1})^{(8)}$ gives the valid bound

$$\left\| C_1^{(7)} \right\|_\infty \le B_7(C_1) := \sum_{k=0}^{8} \binom{8}{k} B_k(y_u) \widetilde{B}_{8-k}(D^{-1}). \tag{5.6}$$

The eighth-order product bound supplies the seventh-order bound for C_1. The reciprocal record is constructed at a higher order, and its tightened bounds through order eight feed this product. These improved intermediate bounds can therefore improve $B_7(R)$, even though tightening does not appear as a separate term in the final Taylor formula. The amount of improvement depends on which candidate bounds are actually smaller.

6 Signed residual assembly and the final estimate

Write the residual as

$$R = \mu Q - y - P - H[yy_u] + \beta C_1 - \beta H[G_u], \tag{6.1}$$

$$D = \sqrt{y_u^2 + (1+Q)^2}, \qquad G = (1+Q)/D, \qquad C_1 = \partial_u(y_u/D). \tag{6.2}$$

The other branches supply their own valid Taylor records. Positivity of D is required for their reciprocal operations. This note analyzes the P branch; it does not establish those other records from scratch.

Theorem 6.1 (Linear assembly of valid records)**.** *Suppose $f_1, \dots, f_s$ have valid records on a common grid and $R = \sum_{\ell=1}^{s} \lambda_\ell f_\ell$, with real constants λ_ℓ. Then a valid record for R is given by*

$$c_{j,m}^R = \sum_\ell \lambda_\ell c_{j,m}^\ell, \qquad e_{j,m}^R = \sum_\ell |\lambda_\ell| e_{j,m}^\ell, \tag{6.3}$$

$$B_m(R) = \sum_\ell |\lambda_\ell| B_m(f_\ell). \tag{6.4}$$

Consequently the order-six residual estimate is

$$\|R\|_\infty \le \max_j \left\{ \sum_{m=0}^{6} U_{j,m}(R) \frac{h^m}{m!} + B_7(R) \frac{h^7}{7!} \right\}, \tag{6.5}$$

provided the residual is periodic.

Proof. Differentiation commutes with the finite linear combination. Subtract the center in (6.3); the triangle inequality bounds the remaining sum of sample errors by $e_{j,m}^R$. Applying the same argument to the exact derivatives proves (6.4). Smoothness and the common positive spacing are preserved. Apply Theorem 4.2 to the resulting record. □

Lean correspondence. `scalar_mul_taylorseries_correct` in `TaylorSeriesCorrectness.lean`.

The same file proves `sub_taylorseries_correct` and `sup_bound_taylorseries_correct`. Their structural composition for $\mu Q - y - P - H[yy_u]$ is `path_a_residual_taylor_correct` in `ResidualAssemblyCorrectness.lean`. Formula (6.3) is in real arithmetic; the numerical code also encloses rounding of the assembled centers and includes it in the output balls.

The order of operations matters for the size of the estimate. The verifier subtracts $I_{j,m}(P)$ into the signed residual derivative interval before taking its absolute bound. It does not add separately computed positive supremum estimates for the nonlinear terms. Thus the centers can retain cancellation among residual terms. The seventh-order global bound still uses the triangle inequality. The product's tightened bounds of orders zero through six play no role in either of these operations.

7 The Julia code path

All file paths in this section are relative to `wave-cap/BabenkoRigorous/src/`. The excerpts use variables from the existing driver; they are not separate executable scripts.

7.1 Fourier construction and local records

In `ResidualBound.jl`, `_residual_bound_full` constructs this branch:

```
y   = cosine_series(y_coeffs; prec=prec)
yu  = deriv(y)
Hyu = hilbert(yu)

M_h = M_order + 2 + extra_orders
ts_y_h = trigpoly_to_taylor(y, N_grid, M_h; prec=prec)
ts_Hyu = trigpoly_to_taylor(Hyu, N_grid, M_order; prec=prec)
ts_y   = truncate_order(ts_y_h, M_order)

ts_y_Hyu = ts_y * ts_Hyu
```

The functions `cosine_series`, `deriv`, `hilbert`, `tp_eval`, and `supnorm_bound` are in `FourierSeries.jl`. The initial high-order record for y is shared with the yy_u branch. `truncate_order` selects the first seven derivative columns and the first eight global bounds for this product; it does not approximate y by a different trigonometric polynomial.

In `TaylorSeries.jl`, `Base.:*` computes (3.2), calls `_leibniz_sup_bounds` for (3.3), constructs the output record, and returns `tighten_sup_bounds!(out)`. The latter consists of `_grid_abs_max_ubs!`, which scans the stored intervals, followed by `_tighten_from_max!`, whose loop runs from `M:-1:0`. The entry `sup_bounds[M+2]`, representing derivative order $M+1$, is untouched.

7.2 The production sharded route

The stage `_stage_p1!` in `Sharded.jl` uses the same row operations on blocks of global grid indices:

```
for si in shard_range
    jr = su.shards[si]
    ts_y_h = trigpoly_to_taylor(su.y, su.N_grid, su.M_h;
                                prec=p, j_range=jr)
    ts_Hyu = trigpoly_to_taylor(su.Hyu, su.N_grid, M_order;
                                prec=p, j_range=jr)
    ts_y = truncate_order(ts_y_h, M_order)
    ts_y_Hyu = ts_y * ts_Hyu
    _acc!(acc_yHyu, ts_y_Hyu.derivs, M_order)
    write_matrix_slices(store, "yHyu", si, ts_y_Hyu.derivs)
end
```

The global grid size, not the shard size, determines u_j and h. The accumulator keeps the largest upper endpoint of the absolute derivative interval at each order. After merging worker accumulators, `_stage_fft!` reconstructs the globally valid bound vector:

```
sb_y_Hyu = _leibniz_sup_bounds(sb_y, sb_Hyu,
                               M_order, p, su.binom_M)
_tighten_from_max!(sb_y_Hyu, st.acc_yHyu.ubs,
                   M_order, su.h_arb, p)
```

Although this stage also performs FFT work for other terms, these two operations for $yH[y_u]$ are scalar bound calculations. The tightening uses maxima over the entire grid. Bounds obtained from a single shard's maxima are not used as certificates over the whole period.

The stage `_stage_s3` reloads the stored rows, forms the residual with the same signs as (6.1), and applies `_supbound_scan`. The code obtains its remainder correction from

```
wR, remR = _taylor_step_weights(M_order, su.h_arb, p)
fboundR = Arblib.ubound(sb_R[M_order + 2] * remR)
```

Here `wR` contains outward evaluations of $h^m/m!$, and `remR` contains $h^{M+1}/(M+1)!$. The scan evaluates the sum of absolute local derivative balls, adds the remainder as a radius using `add_error!`, and compares upper endpoints across cells. The returned Arb ball's upper endpoint is the certificate. Its midpoint alone is not the residual bound.

7.3 Cost of this branch

At order six the local product has $\sum_{m=0}^{6}(m+1) = 28$ Leibniz summands per row, or 56 million over the full grid. One reduction across the seven derivative columns visits 14 million entries. The scalar seed vector has 36 Leibniz summands over orders $0, \dots, 7$, and one downward scalar tightening pass has seven steps. The sharded driver performs both local product reductions and accumulator reductions; these counts do not claim that only one such scan occurs in the whole program. The archived run does not provide a separate timer for $yH[y_u]$, so no isolated wall time is inferred here. The full product matrix need not be resident in memory at once.

8 What the dedicated Lean certificate establishes

The general analytic declarations described above have ordinary mathematical proofs. Their hypotheses must still be connected to numerical data. In particular, existence of a valid Taylor record does not establish that a particular serialized Arb array is that record. The product correctness theorem explicitly assumes its sample-error inequality, and the grid-to-continuum theorem explicitly assumes its cellwise bound.

The dedicated certificate uses `CertificatePhase4ScalarN150Phase420260827HalfDC.lean` for exact candidate constants and endpoints. Its numerical interface is in `CertificatePhase4ScalarN150Phase420260827HalfDCAxioms.lean`. It assumes the following two statements, rather than proving them by replaying the two-million-point computation in Lean:

$$d_* > 0, \qquad D(u) \geq d_* \quad (u \in \mathbb{R}), \tag{8.1}$$

$$|R(u)| \leq \varepsilon_* \quad (u \in [0, 2\pi]), \quad \text{given (8.1)}. \tag{8.2}$$

They are named `cert_arb_dmin_bound` and `cert_arb_sup_bound_le_eps0`, respectively. The archived upper endpoint is approximately $1.7792768524432498 \times 10^{-13}$, and the denominator lower endpoint is approximately 0.34465883518530394. These are bounds for the full residual and denominator, not an isolated numerical bound on $\|P\|_\infty$.

Theorem 8.1 (Global consequence of the numerical certificate)**.** *Assume* (8.1)–(8.2) *for the certified candidate. Then* $D(u) \geq d_*$ *and* $|R(u)| \leq \varepsilon_*$ *for every real* u*. For the exact constants in the dedicated certificate, this implies*

$$|R(u)| \leq 1.78 \times 10^{-13}, \qquad D(u) \geq 0.3446588 \qquad (u \in \mathbb{R}). \tag{8.3}$$

Proof. The denominator assertion is already part of (8.1). The residual is 2π-periodic: its factors, derivatives, and periodic Hilbert transforms have that period. Choose a representative of u modulo 2π in $[0, 2\pi]$ and apply (8.2) there. For the decimal statements, compare the exact rational certificate endpoints with $178/10^{15}$ and $3446588/10^7$, respectively. □

The periodic extension is `cert_residual_le` in the dedicated axioms module. The decimal comparisons are proved by `residual_bounded_by_phase4_n150_half_dc_decimal` and `D_bounded_below_by_phase4_n150_half_dc_decimal` in `MainPhase4ScalarN150Phase420260827HalfDC.lean`. The two numerical assumptions remain assumptions of these conclusions; the displayed proof does not replace their interval-arithmetic verification.

For comparison, the generic record construction in `CertificateAxioms.lean` uses the product-existence theorem in the form

```
cert_arb_ts_mul h_y h_Hyu (hh_y.trans hh_Hyu.symm)
```

where `h_y`, `h_Hyu` establish valid factor records and `hh_y`, `hh_Hyu` identify their spacings with $2\pi/n$. This generic construction is not the numerical dependency chain of the dedicated half-DC theorem.

9 Source correspondence

The files listed below are part of the repository snapshot used for this note. Julia paths are relative to `wave-cap/BabenkoRigorous/src/`; Lean paths are relative to `wave-cap/WaveCAP/WaveCAP/`. The mathematical proofs in this note are readable presentations and stated specializations of the Lean declarations, not new Lean source files. Corollary 5.3 and Proposition 5.5 are explicitly identified implementation consequences rather than claims of separately formalized declarations.

The referenced modules build with Lean `v4.29.0-rc6`. An axiom audit of the listed analytic declarations reports only `propext`, `Classical.choice`, and `Quot.sound`. The dedicated numerical conclusions additionally depend on the assumptions identified in Section 8.

Purpose	File and principal declarations
Fourier algebra	`FourierSeries.jl`: `cosine_series`, `deriv`, `hilbert`, `tp_eval`, `supnorm_bound`.
Taylor arithmetic	`TaylorSeries.jl`: `trigpoly_to_taylor`, `truncate_order`, `Base.:*`, `_leibniz_sup_bounds`, `tighten_sup_bounds!`, `sup_bound`.
Residual implementation	`ResidualBound.jl` and `Sharded.jl`: `_residual_bound_full`, `_stage_p1!`, `_stage_fft!`, `_stage_s3`.
Hilbert identity	`HilbertOnTrigPoly.lean`: `periodicHilbert_yu_eq_Hyu`.
Coefficient convention	`HalfDCConvention.lean`: `yHalfDC_apply`, `yu_halfDCCoeffs`, `Hyu_halfDCCoeffs`.
Product bound	`LeibnizBound.lean`: `abs_iteratedDeriv_mul_le`.
Record correctness	`TaylorSeriesCorrectness.lean`: `trigpoly_to_taylor_correct`, `mul_taylorseries_correct`, `sup_bound_taylorseries_correct`.
Taylor theorem	`TaylorRemainder.lean`: `abs_taylor_remainder_le`; `GridToSupNormTaylor.lean`: `grid_to_supNorm_taylor`.
Grid tightening	`TightenSupBoundsLemma.lean`: `grid_lipschitz_sup_bound_step`, `tighten_sup_bounds_correct`, `tighten_sup_bounds_taylorseries_correct`.
Generic existence	`CertificateAxioms.lean`: `exists_isTaylorSeriesOf_of_contDiff_periodic`, `cert_arb_trigpoly_to_taylor`, `cert_arb_ts_mul`.
Residual assembly	`ResidualAssemblyCorrectness.lean`: `path_a_residual_taylor_correct`.